

Module 1:

1. Write a program to define 3 different threads with the following purpose where N is the input

- Thread A - To run a loop and return the sum of first N prime numbers.
- Thread B and C – should run in parallel. One prints “Thread 1 running every 2 second”, and the other prints “Thread 2 running every 3 second”

Code :

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

int N;

int isPrime(int num){
    if (num < 2) return 0;
    for (int i = 2; i * i <= num; i++){
        if (num % i == 0)
            return 0;
    }

    return 1;
}

void *threadA(void *arg){
    int count = 0;
    int num = 2;
    int sum = 0;

    while (count < N){
        if (isPrime(num)){
            sum += num;
            count++;
        }
        num++;
    }

    printf("\nSum of first %d prime numbers = %d\n", N, sum);

    pthread_exit(NULL);
}
```

```
void *threadB(void *arg){
    int time = 0;

    while (time < 100){
        printf("Thread 1 running every 2 seconds\n");
        sleep(2);
        time += 2;
    }

    pthread_exit(NULL);
}

void *threadC(void *arg){
    int time = 0;

    while (time < 100){
        printf("Thread 2 running every 3 seconds\n");
        sleep(3);
        time += 3;
    }

    pthread_exit(NULL);
}

int main(){
    pthread_t t1, t2, t3;

    printf("Enter N: ");
    scanf("%d", &N);

    pthread_create(&t1, NULL, threadA, NULL);
    pthread_create(&t2, NULL, threadB, NULL);
    pthread_create(&t3, NULL, threadC, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    return 0;
}
```

Output :

```
PS E:\college\embed\_Virtual training\adv\Adv_C\mod2> gcc m2q1.c -o m2q1.exe
PS E:\college\embed\_Virtual training\adv\Adv_C\mod2> ./m2q1.exe
Enter N: 5

Sum of first 5 prime numbers = 28
Thread 1 running every 2 seconds
Thread 2 running every 3 seconds
Thread 1 running every 2 seconds
Thread 2 running every 3 seconds
Thread 1 running every 2 seconds
Thread 2 running every 3 seconds
Thread 1 running every 2 seconds
Thread 1 running every 2 seconds
Thread 2 running every 3 seconds
Thread 1 running every 2 seconds
Thread 2 running every 3 seconds
Thread 1 running every 2 seconds
```

2. In the above program,

a) Add signal handler for SIGINT (etc) and prevent termination.

Code :

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <signal.h>

int N;

int isPrime(int num){
    if (num < 2) return 0;
    for (int i = 2; i * i <= num; i++){
        if (num % i == 0)
            return 0;
    }

    return 1;
}

void *threadA(void *arg){
    int count = 0;
    int num = 2;
    int sum = 0;

    while (count < N){
        if (isPrime(num)){
            sum += num;
            num++;
        }
        count++;
    }
}
```

```

        sum += num;
        count++;
    }
    num++;
}

printf("\nSum of first %d prime numbers = %d\n", N, sum);

pthread_exit(NULL);
}

void *threadB(void *arg){
    int time = 0;

    while (time < 100){
        printf("Thread 1 running every 2 seconds\n");
        sleep(2);
        time += 2;
    }

    pthread_exit(NULL);
}

void *threadC(void *arg){
    int time = 0;

    while (time < 100){
        printf("Thread 2 running every 3 seconds\n");
        sleep(3);
        time += 3;
    }

    pthread_exit(NULL);
}

void signalHandler(int sig)
{
    printf("\nSIGINT received. Program will not terminate.\n");
    fflush(stdout);
}

int main(){

    signal(SIGINT, signalHandler);
    pthread_t t1, t2, t3;

```

```

printf("Enter N: ");
scanf("%d", &N);

pthread_create(&t1, NULL, threadA, NULL);
pthread_create(&t2, NULL, threadB, NULL);
pthread_create(&t3, NULL, threadC, NULL);

pthread_join(t1, NULL);
pthread_join(t2, NULL);
pthread_join(t3, NULL);
return 0;
}

```

Output :

```

E:\college\embed\_Virtual training\adv\Adv_C\mod2>m2q2.exe
Enter N: 15
Thread 1 running every 2 seconds

Sum of first 15 prime numbers = 328
Thread 2 running every 3 seconds
Thread 1 running every 2 seconds

SIGINT received. Program will not terminate.
Thread 2 running every 3 seconds
Thread 1 running every 2 seconds

```

b) Convert the above threads to individual functions and note down the time taken and the flow of execution.

Code :

```

#include <stdio.h>
#include <unistd.h>

int N;

int isPrime(int num)
{
    if(num < 2)
        return 0;

    for(int i = 2; i * i <= num; i++)
    {

```

```
        if(num % i == 0)
            return 0;
    }

    return 1;
}

void functionA()
{
    int count = 0;
    int num = 2;
    int sum = 0;

    while(count < N)
    {
        if(isPrime(num))
        {
            sum += num;
            count++;
        }
        num++;
    }

    printf("Sum of first %d prime numbers = %d\n", N, sum);
}

void functionB()
{
    int time = 0;

    while(time < 10)
    {
        printf("Function B running every 2 seconds\n");
        sleep(2);
        time += 2;
    }
}

void functionC()
{
    int time = 0;

    while(time < 10)
```

```

    {
        printf("Function C running every 3 seconds\n");
        sleep(3);
        time += 3;
    }
}

int main()
{
    printf("Enter N: ");
    scanf("%d", &N);

    functionA();
    functionB();
    functionC();

    return 0;
}

```

Output :

```

PS E:\college\embed\_Virtual training\adv\Adv_C\mod2> gcc m2qb.c -o m2qb.exe
PS E:\college\embed\_Virtual training\adv\Adv_C\mod2> ./m2qb.exe
Enter N: 5
Sum of first 5 prime numbers = 28
Function B running every 2 seconds
Function B running every 2 seconds
Function B running every 2 seconds
Function B running every 2 seconds
Function B running every 2 seconds
Function C running every 3 seconds

```

Here the function B is running first and only after its execution C is run, thus making the runtime twice than it was earlier.

3. Know about the following topics and explore them (Write a note on your understandings)

Area for exploration,

a) Child Process – fork()

b) Handling Common Signals

c) Time Complexity

d) Locking Mechanism – Mutex and spin lock

a) Child Process – fork()

- `fork()` is a system call used in Linux/Unix to create a new process.
- The new process created is called the child process and the original process is called the parent process.
- After `fork()`, both parent and child execute independently.
- The child process gets a copy of the parent's memory space.
- `fork()` returns 0 in the child process and the child PID in the parent process.
- It is commonly used to create multiple processes for multitasking.

b) Handling Common Signals

- Signals are software interrupts used to notify a process about an event.
- They help processes respond to external or internal events.
- A signal handler can be used to catch and process signals.
- Common signals include:
 - `SIGINT` – Generated when Ctrl + C is pressed.
 - `SIGTERM` – Requests process termination.
 - `SIGKILL` – Forcefully terminates a process.
 - `SIGSEGV` – Occurs during invalid memory access.
- Signal handling improves program reliability and allows custom actions before termination.

c) Time Complexity

- Time complexity describes how the execution time of an algorithm grows with input size.
- It helps compare the efficiency of different algorithms.
- Big O notation is commonly used to represent time complexity.
- Common complexities:
 - $O(1)$ – Constant time
 - $O(\log n)$ – Logarithmic time
 - $O(n)$ – Linear time
 - $O(n \log n)$ – Efficient sorting algorithms
 - $O(n^2)$ – Nested loops
- Lower time complexity generally means better performance for large inputs.

d) Exploring Different Kernel Crashes

- A kernel crash occurs when the operating system kernel encounters a critical error.
- Since the kernel controls hardware and system resources, a crash can stop the entire system.
- Common causes include:
 - Invalid memory access
 - Faulty device drivers
 - Hardware failures
 - Kernel bugs
- In Windows, failures may result in a Blue Screen of Death (BSOD).

e) Locking Mechanism – Mutex and Spin Lock

- Locking mechanisms are used to prevent multiple threads from accessing shared resources simultaneously.
- They help avoid race conditions and maintain data consistency.

Mutex (Mutual Exclusion)

- A mutex allows only one thread to access a critical section at a time.
- Other threads are blocked until the mutex is released.
- Suitable when the waiting time may be longer.
- Commonly used in multithreaded applications.

Spin Lock

- A spin lock continuously checks until the lock becomes available.
- Instead of sleeping, the thread keeps "spinning" in a loop.
- Suitable for very short waiting periods.
- Frequently used inside operating system kernels where blocking is expensive.

Mutex	Spin Lock
Waiting thread sleeps	Waiting thread keeps checking
Better for longer waits	Better for short waits
Lower CPU usage	Higher CPU usage
Used in user applications	Common in kernel programming